ИНФОРМАТИКА · 10-СЫНЫП

РЕКУРСИЯ

Функцияның өзін-өзі шақыруы

Оқу мақсаты: 10.5.1.2 — жолдарды өңдеу үшін процедуралар мен функцияларды пайдалану

Орындаған: Тұрдалы Ақтілек

Сабақ мақсаты

Сабақ соңында оқушы:

01 **ТҮСІНЕДІ** Рекурсия ұғымын және оның не үшін қолданылатынын түсіндіреді.

02 **АЖЫРАТАДЫ** Базалық жағдай мен рекурсивті қадамды анықтайды.

03 **ҚОЛДАНАДЫ** Python тілінде жолды рекурсивті өңдейтін функция жазады.

Рекурсия дегеніміз не?

Негізгі анықтама

Рекурсия — функцияның есепті шешу үшін өзін-өзі қайта шақыру тәсілі.

- Үлкен есеп ұқсас, кішірек есептерге бөлінеді.
- Әр шақыруда есеп базалық жағдайға жақындауы керек.
- Рекурсия жолдарды, ағаш құрылымдарын және бөлшектенетін есептерді өңдеуге көмектеседі.

Күнделікті өмірдегі ұқсастық

Бір әрекет кішірейіп қайталанады

3

3 санын айт ⟶ 2-ге өт

⟶

2

2 санын айт ⟶ 1-ге өт

⟶

1

1 санын айт ⟶ тоқта

Бағдарламада да әр шақыру есепті азайтып, тоқтау шартына жетуі қажет.

Рекурсияның 2 міндетті бөлігі

Екеуі де дұрыс құрылуы керек

Базалық жағдай

Рекурсияны тоқтатады.

Мысалы: if n == 0: return

Рекурсивті қадам

Функцияны кішірейген мәнмен қайта шақырады.

Мысалы: return f(n - 1)

Базалық жағдай болмаса, шақырулар шектен тыс жалғасуы мүмкін.

Мысал 1. Кері санау

Әр қадамда n біреуге азаяды

```
def countdown(n):
    if n == 0:
        print("Бітті!")
    else:
        print(n)
        countdown(n - 1)

countdown(3)
```

Орындалу реті

countdown(3) ⟶ 3

countdown(2) ⟶ 2

countdown(1) ⟶ 1

countdown(0) ⟶ Бітті!

Нәтиже: 3, 2, 1, Бітті!

Жолды өңдеу және рекурсия

Оқу мақсатына тікелей байланыс

- Жол (string) — таңбалар тізбегі. Мысалы: "Python".
- s[0] — бірінші таңба; s[1:] — бірінші таңбадан кейінгі бөлік.
- len(s) — жолдың ұзындығын береді.
- Рекурсия әр шақыруда жолдың бір бөлігін өңдеп, қалған бөлігіне қайта шақыру жасай алады.

Маңызды: жол бос болғанда немесе бір таңба қалған кезде тоқтау шартын анықта.

Мысал 2. Жолды кері айналдыру

reverse_text("Python") ⟶ "nohtyP"

```
def reverse_text(s):
    if len(s) <= 1:
        return s
    return reverse_text(s[1:]) + s[0]

print(reverse_text("cat"))
```

Қадамдар: 'cat'

reverse("cat")

= reverse("at") + "c"

= reverse("t") + "a" + "c"

= "tac"

Базалық жағдай: len(s) ≤ 1

Мысал 3. Жолдағы таңбаларды санау

Базалық жағдай: бос жол ⟶ 0

```
def count_chars(s):
    if s == "":
        return 0
    return 1 + count_chars(s[1:])

print(count_chars("code"))
```

Талдау

"code" ⟶ 1 + count("ode")

"ode" ⟶ 1 + count("de")

"de" ⟶ 1 + count("e")

"e" ⟶ 1 + count("")

"" ⟶ 0

Жауабы: 4

Рекурсияны қадам бойынша бақылау

Мысал: countdown(3)

Шақыру	Шарт / әрекет	Нәтиже
countdown(3)	3 != 0, print(3)	3
countdown(2)	2 != 0, print(2)	2
countdown(1)	1 != 0, print(1)	1
countdown(0)	n == 0	Бітті!

Әр шақыру жеке орындалады; базалық жағдайға жеткенде жаңа шақыру жасалмайды.

Рекурсивті функция құру алгоритмі

Жазу алдында осы 4 қадамды тексер

1. Есептің ең қарапайым жағдайын анықта.
2. Базалық жағдай үшін қайтарылатын мәнді жаз.
3. Қалған есепті кішірейтіп, функцияны қайта шақыр.
4. Кемінде 2 тестпен тексер: бос жол, бір таңба немесе қалыпты жол.

Артықшылықтары және сақ болатын тұстары

Рекурсияны орынды қолдан

✓ Артықшылықтары

- Қайталанатын құрылымды ықшам сипаттайды.
- Кодты кейбір есептерде түсінікті етеді.
- Жолдың бөліктерін біртіндеп өңдеуге болады.

! Назар аудар

- Базалық жағдайды ұмытпа.
- Әр қадамда есеп кішіреюі керек.
- Тым терең шақыру Python-да RecursionError қатесін тудыруы мүмкін.

Қарапайым қайталанатын есептерде цикл кейде тиімдірек болуы мүмкін.

Практикалық тапсырма

Жеке және жұптық жұмыс

1 · Кодты болжа

f(3) функциясының нәтижесін кодты іске қоспай анықта.

2 · Жолды кері жаз

reverse_text("Python") функциясын іске қосып, нәтижесін түсіндір.

3 · Таңбаларды сана

count_chars("hello") нәтижесін тап және базалық жағдайды ата.

Бағалау: дұрыс нәтиже — 1 балл; базалық жағдайды түсіндіру — 1 балл; кодты тестілеу — 1 балл.

Жылдам тексеру

Ауызша жауап бер немесе дәптерге жаз

- 1. Рекурсия дегеніміз не?
- 2. Базалық жағдай қандай қызмет атқарады?
- 3. Рекурсивті қадамда не өзгеруі керек?
- 4. Жолдың бірінші таңбасын Python-да қалай аламыз?
- 5. Рекурсия тоқтамаса қандай мәселе туындауы мүмкін?

Есте сақта!

Сабақтың негізгі қорытындысы

Рекурсия = базалық жағдай ✦ кішірейетін рекурсивті қадам

- Функция өзін-өзі шақырады.
- Базалық жағдай тоқтауды қамтамасыз етеді.
- Жолдарды өңдеу үшін s[0], s[1:] және len(s) пайдалы.
- Кодты іске қосып, бірнеше тестпен тексер.

Кері байланыс: «Бүгін мен түсіндім… / Маған әлі де қиын…»

Пайдаланылған дереккөздер

Түсіндірмелер мен Python синтаксисін тексеруге арналған

- Python.org — ресми Python құжаттамасы: https://www.python.org/doc/
- Python Tutorial — функциялар және функция шақырулары: https://docs.python.org/3/tutorial/controlflow.html
- Оқу мақсаты: 10.5.1.2 — жолдарды өңдеу үшін процедуралар мен функцияларды пайдалану.

Ескерту: мысалдар оқу мақсатында берілген; Python 3 ортасында орындауға болады.

Негізгі ұғымдардың нақты анықтамалары

Терминдерді шатастырмау үшін

Рекурсия

Функцияның есепті шешу барысында өзін-өзі қайта шақыруы.

Рекурсивті функция

Өз денесінің ішінде өзін қайта шақыратын функция.

Базалық жағдай

Функция қайта шақырылмай, нәтиже беретін тоқтау шарты.

Рекурсивті қадам

Есепті кішірейтіп, сол функцияны қайта шақыратын бөлік.

Жол (string)

Реті сақталған таңбалар тізбегі. Мысалы: "Python".

Қайтарылатын мән (return)

Функция есептеген нәтижені шақырылған орнына қайтарады.

Функция және жолды өңдеу: нақтылау

Python мысалдары

Функция

Функция — атауы бар код бөлігі; оны шақырғанда белгілі бір әрекетті орындайды және қажет болса мән қайтарады.

Мысал:

```
def square(n):
    return n * n
```

Жолды бөліктерге бөлу

s[0] — бірінші таңба.

s[1:] — бірінші таңбадан кейінгі бөлік.

len(s) — таңбалар саны.

Мысал: s = "cat"

s[0] ⟶ "c"; s[1:] ⟶ "at"

Процедура — әрекет орындайтын код бөлігі; Python-да ол мән қайтармайтын функция ретінде жазылуы мүмкін.

Рекурсияның жұмыс істеу логикасы

Мысал: count_chars("cat")

1	**count_chars('cat')**	1 + count_chars("at")
2	**count_chars('at')**	1 + count_chars("t")
3	**count_chars('t')**	1 + count_chars("")
4	**count_chars('')**	0 — базалық жағдай

Кері қайту кезінде: 1 ✦ 1 ✦ 1 ✦ 0 = 3. Демек, 'cat' жолында 3 таңба бар.